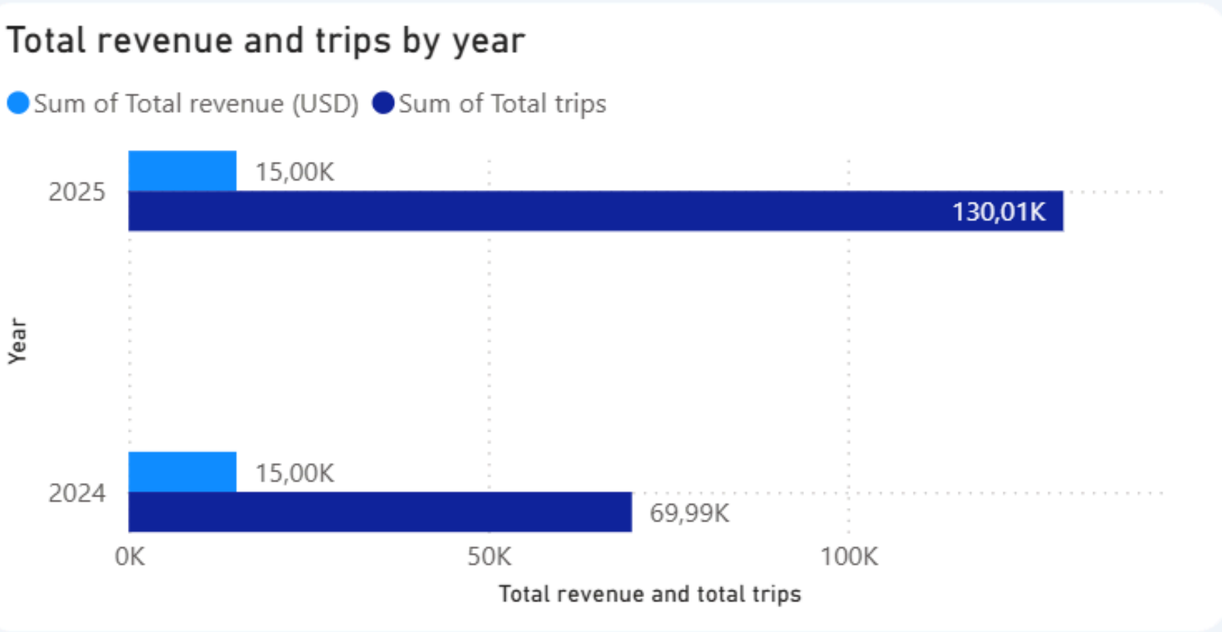
Financial Project : SQL



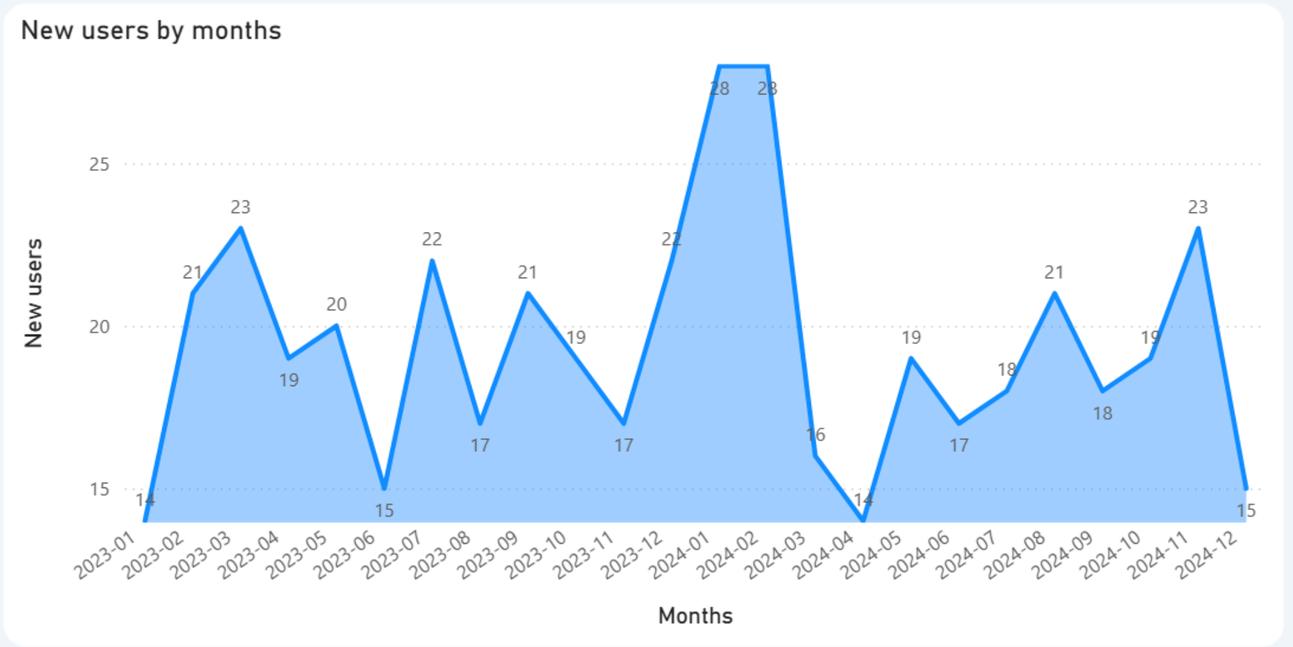
Question 1: what is the estimated annual revenue ?

```sql
SELECT
    YEAR(t.start_datetime) AS year,
    COUNT(DISTINCT t.user_id) AS active_subscribers,
    COUNT(*) AS total_trips,
    COUNT(DISTINCT t.user_id) * 30 AS total_revenue_est_usd
FROM Trips t
GROUP BY YEAR(t.start_datetime)
ORDER BY year;
```



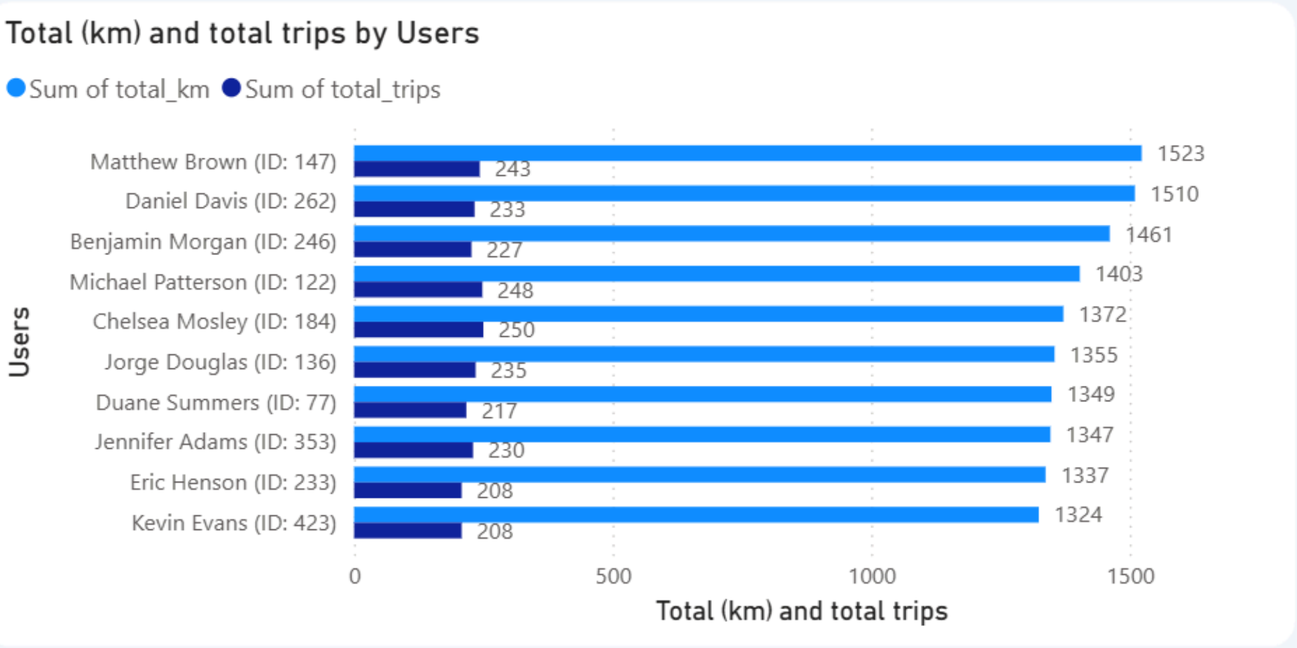
Question 2: what is the monthly subscriber growth trend, and which periods show the highest rate of new user acquisition?

```sql
SELECT
    DATE_FORMAT(registration_date, '%Y-%m') AS month,
    COUNT(*) AS new_users
FROM Users
GROUP BY DATE_FORMAT(registration_date, '%Y-%m');
```



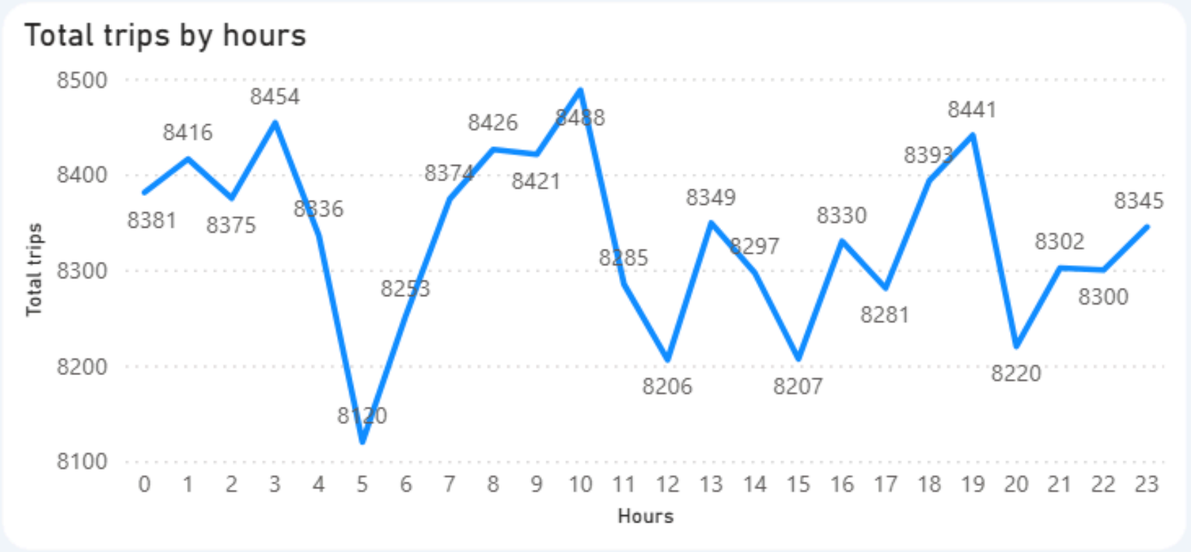
Question 3 : who are our top 10 most active users in terms of total distance traveled?

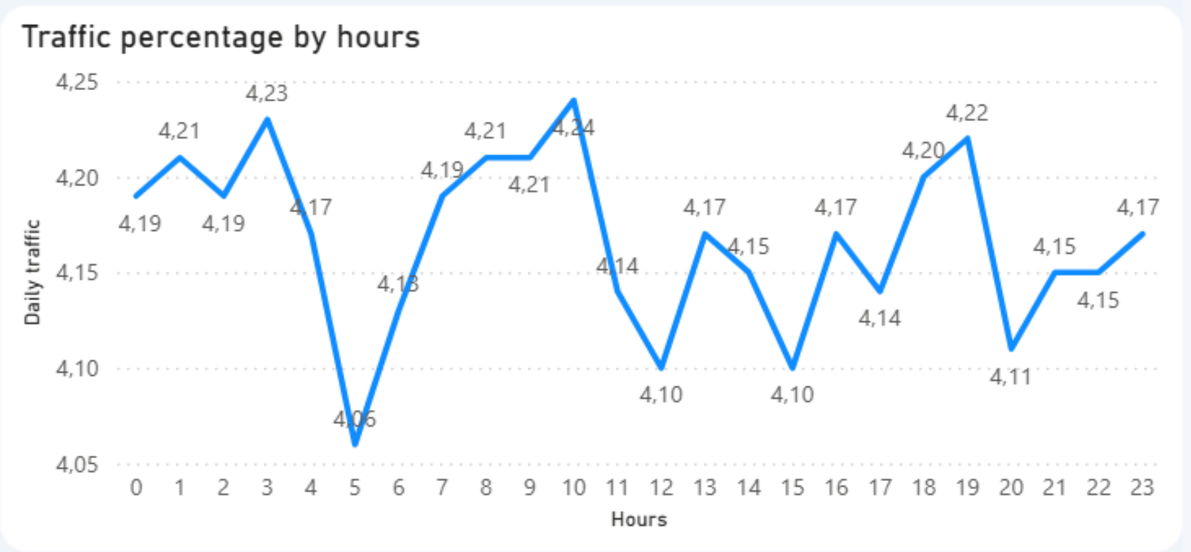
```sql
SELECT
    id,
    first_name,
    last_name,
    total_km,
    total_trips,
    ROUND(total_km / total_trips, 2) AS avg_km_per_trip -
FROM users
WHERE total_km < 100000 -- removing potential tracking er
ORDER BY total_km DESC
LIMIT 10;
```



Question 4: at what specific hours of the day does Divvy experience the highest demand for bike trips?

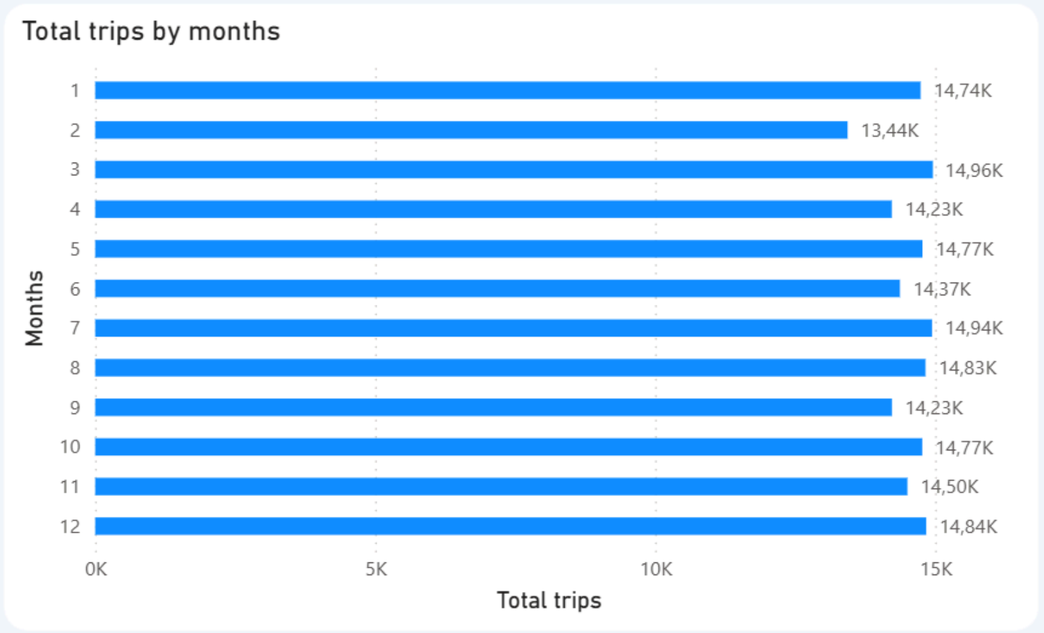
```sql
USE bikeshare_en;
SELECT
    HOUR(start_datetime) AS hour_of_day,
    COUNT(*) AS total_trips,
    -- Calculating the percentage of total daily traffic per hour
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM trips), 2) AS percentage_of_daily_traffic
FROM trips
GROUP BY hour_of_day
ORDER BY hour_of_day;
```

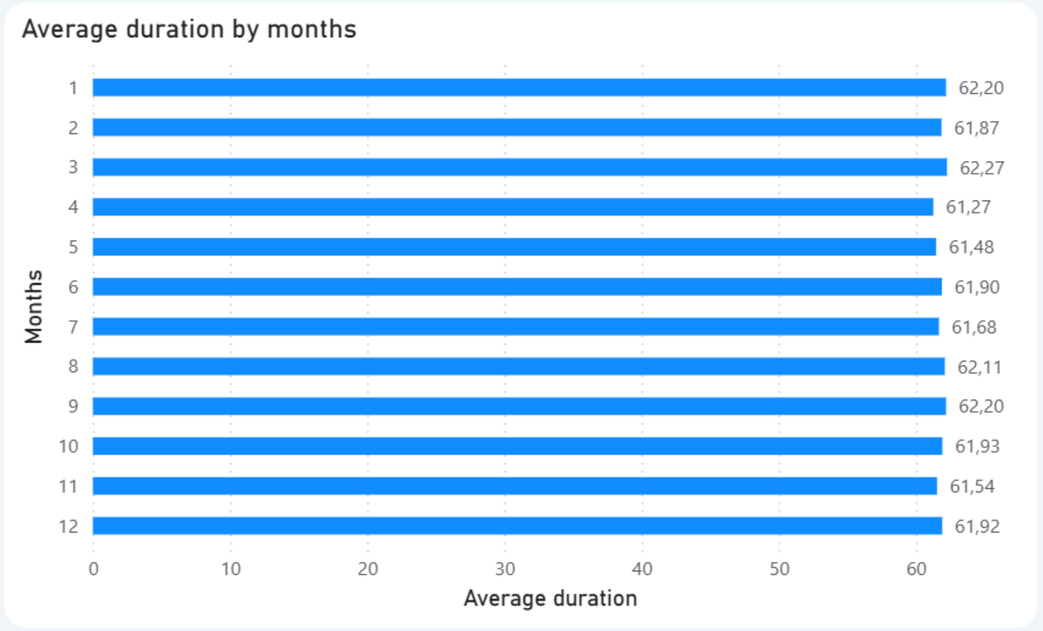




Question 5: how does bike usage fluctuate across different months of the year?

```sql
USE bikeshare_en;
SELECT
    MONTH(start_datetime) AS trip_month,
    COUNT(*) AS total_trips,
    ROUND(AVG(duration_minutes), 2) AS avg_duration --
FROM trips
WHERE duration_minutes > 0 AND duration_minutes < 1440
GROUP BY trip_month
ORDER BY trip_month;
```

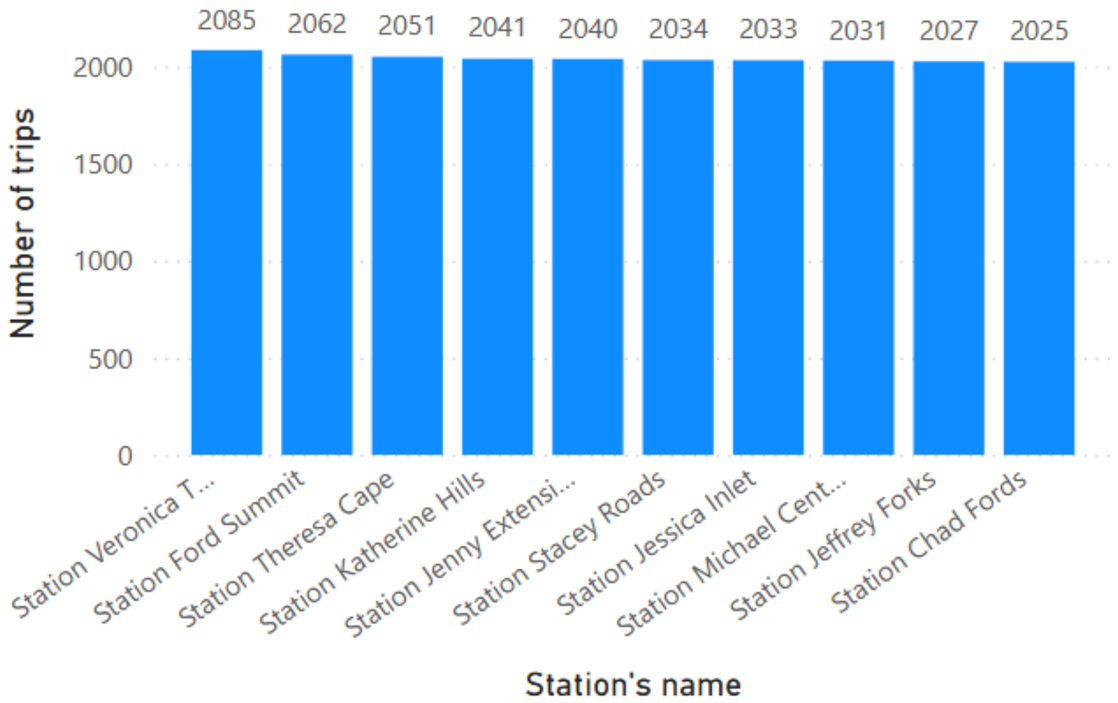




Question 6: which stations are the most used in the bikeshare network?

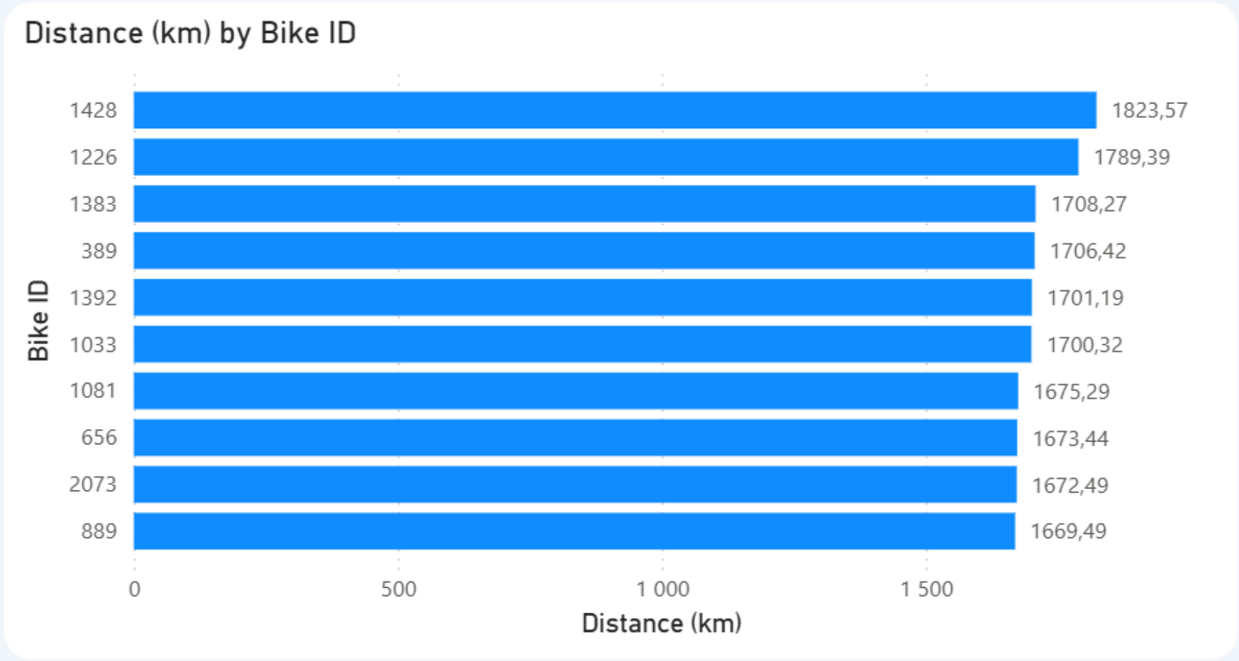
```sql
-- This query identifies the most popular bike stations
-- Popularity is defined as the number of trips starting at each station

-- Q6 - Which stations are the most used in the bikeshare network?
-- popular station = a station with a high number of trip departures
SELECT
    s.id,                       -- Unique identifier of the station
    s.name,                     -- Station name
    COUNT(t.id) AS nb_trips     -- Number of trips starting at the station
FROM trips t
JOIN stations s
    ON t.start_station_id = s.id    -- Join trips with stations using start station
WHERE t.start_station_id IS NOT NULL
    -- Exclude trips without a valid start station
GROUP BY s.id, s.name
    -- Group results by station to aggregate trip counts
ORDER BY nb_trips DESC
    -- Sort stations from most to least popular
LIMIT 10;
    -- Keep only the top 10 most popular stations
```



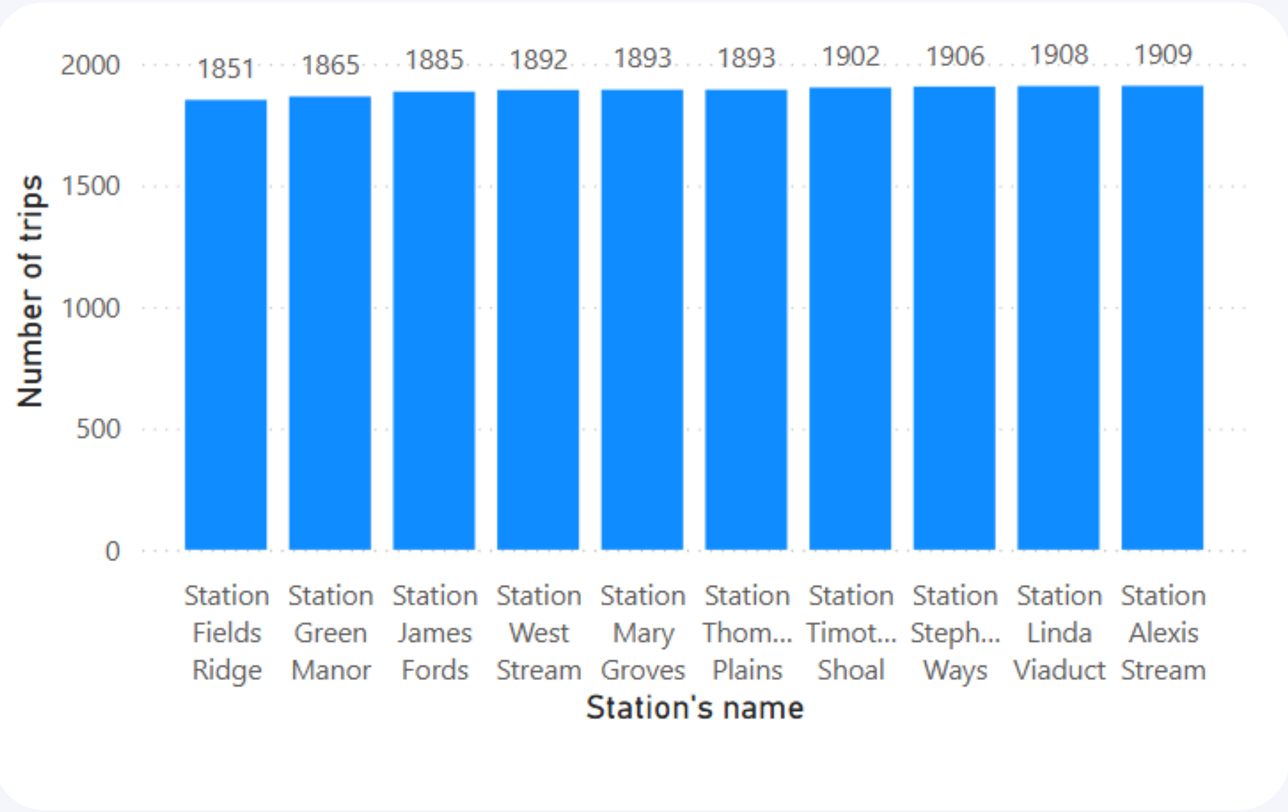
Question 7 which specific bikes (by bike ID) have traveled the most distance and are likely in need of a safety inspection?:

```sql
USE bikeshare_en;
SELECT
    bike_id,
    ROUND(SUM(km), 2) AS cumulative_distance_km, --
    COUNT(id) AS usage_count,                    --
    ROUND(AVG(km), 2) AS avg_distance_per_trip
FROM trips
WHERE km > 0 AND km < 500 -- Data cleaning: focusing
GROUP BY bike_id
ORDER BY cumulative_distance_km DESC
LIMIT 10;
```



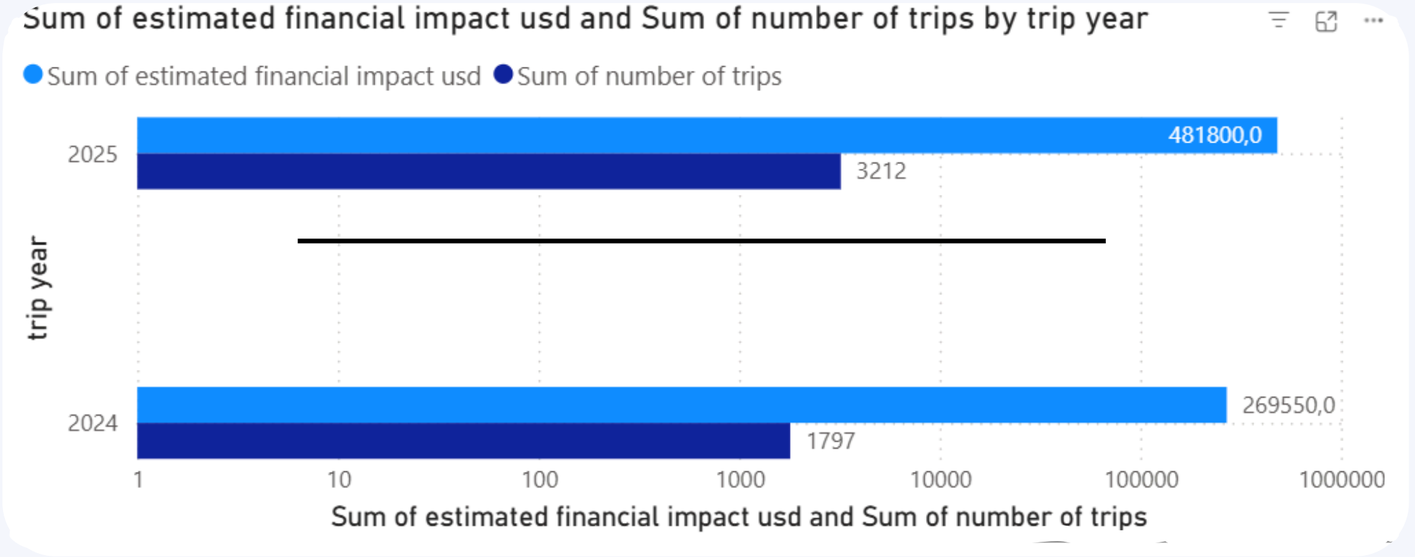
Question 8: Which stations are significantly underused and may represent inefficient assets?

```sql
-- Summing distances for each bike while filtering
-- Business goal: to create a priority list for saf
USE bikeshare_en;
SELECT
    bike_id,
    ROUND(SUM(km), 2) AS cumulative_distance_km, --
    COUNT(id) AS usage_count,                    --
    ROUND(AVG(km), 2) AS avg_distance_per_trip
FROM trips
WHERE km > 0 AND km < 500 -- Data cleaning: focusin
GROUP BY bike_id
ORDER BY cumulative_distance_km DESC
LIMIT 10;
```



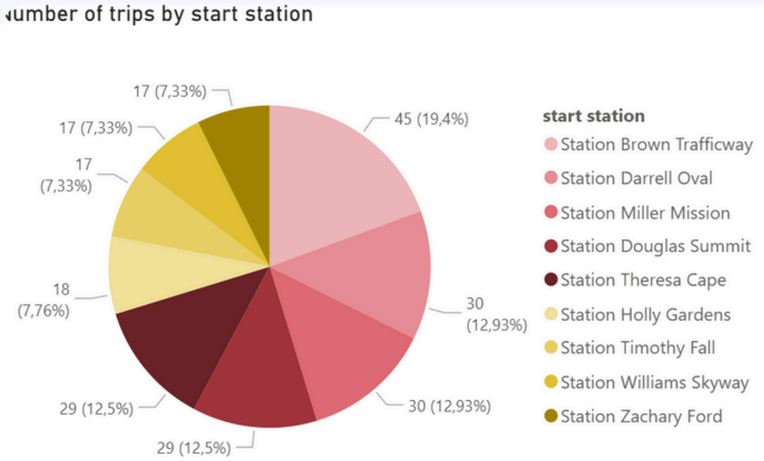
Question 9: how many trips last more than 4 hours (240 minutes)? This could indicate theft or an improperly returned bike.

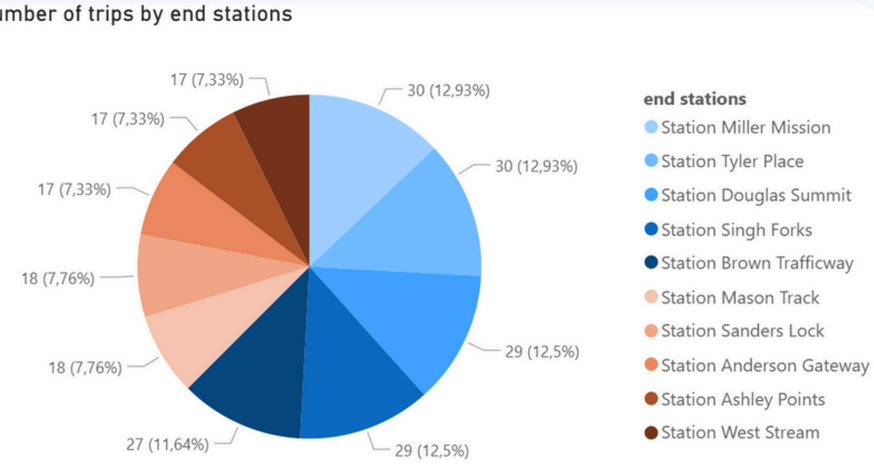
```sql
SELECT
    trip_year,
    trip_category,
    COUNT(*) AS number_of_trips,
    CASE
        WHEN trip_category = 'Lost / Stolen / System Error (Max Value)' THEN COUNT(*) * 150.00
        ELSE SUM(duration_minutes) * 0.15
    END AS estimated_financial_impact_usd
FROM (
    SELECT
        YEAR(start_datetime) AS trip_year,
        duration_minutes,
        CASE
            WHEN duration_minutes >= 9999 THEN 'Lost / Stolen / System Error (Max Value)'
            WHEN duration_minutes > 1440 THEN 'Extreme Delay (> 24 Hours)'
            ELSE 'Long Trip (4 to 24 Hours)'
        END AS trip_category
    FROM trips
    WHERE duration_minutes > 240
) AS subquery_table
GROUP BY trip_year, trip_category
ORDER BY trip_year ASC, number_of_trips DESC;
```



Question 10: what are the most frequent routes (Origin -> Destination)?"

```sql
SELECT * FROM (
    SELECT
        YEAR(t.start_datetime) AS trip_year,
        s1.name AS start_station,
        s2.name AS end_station,
        COUNT(*) AS trip_count,
        ROW_NUMBER() OVER (
            PARTITION BY YEAR(t.start_datetime)
            ORDER BY COUNT(*) DESC
        ) as ranking
    FROM trips t
    JOIN stations s1 ON t.start_station_id = s1.id
    JOIN stations s2 ON t.end_station_id = s2.id
    WHERE
        s1.name IS NOT NULL
        AND s2.name IS NOT NULL
    GROUP BY
        trip_year,
        start_station,
        end_station
) AS ranked_table
WHERE ranking <= 5
ORDER BY trip_year ASC, ranking ASC;
```





start station	end station	Sum of trip count	Sum of trip year
Station Darrell Oval	Station Tyler Place	30	2025
Station Miller Mission	Station Miller Mission	30	2025
Station Douglas Summit	Station Douglas Summit	29	2025
Station Theresa Cape	Station Singh Forks	29	2025
Station Brown Trafficway	Station Brown Trafficway	27	2025
Station Brown Trafficway	Station Sanders Lock	18	2024
Station Holly Gardens	Station Mason Track	18	2024
Station Timothy Fall	Station Anderson Gateway	17	2024
Station Williams Skyway	Station Ashley Points	17	2024
Station Zachary Ford	Station West Stream	17	2024
Total		**232**	**20245**

Question 11 : Which stations have the highest proportion of defective docks?

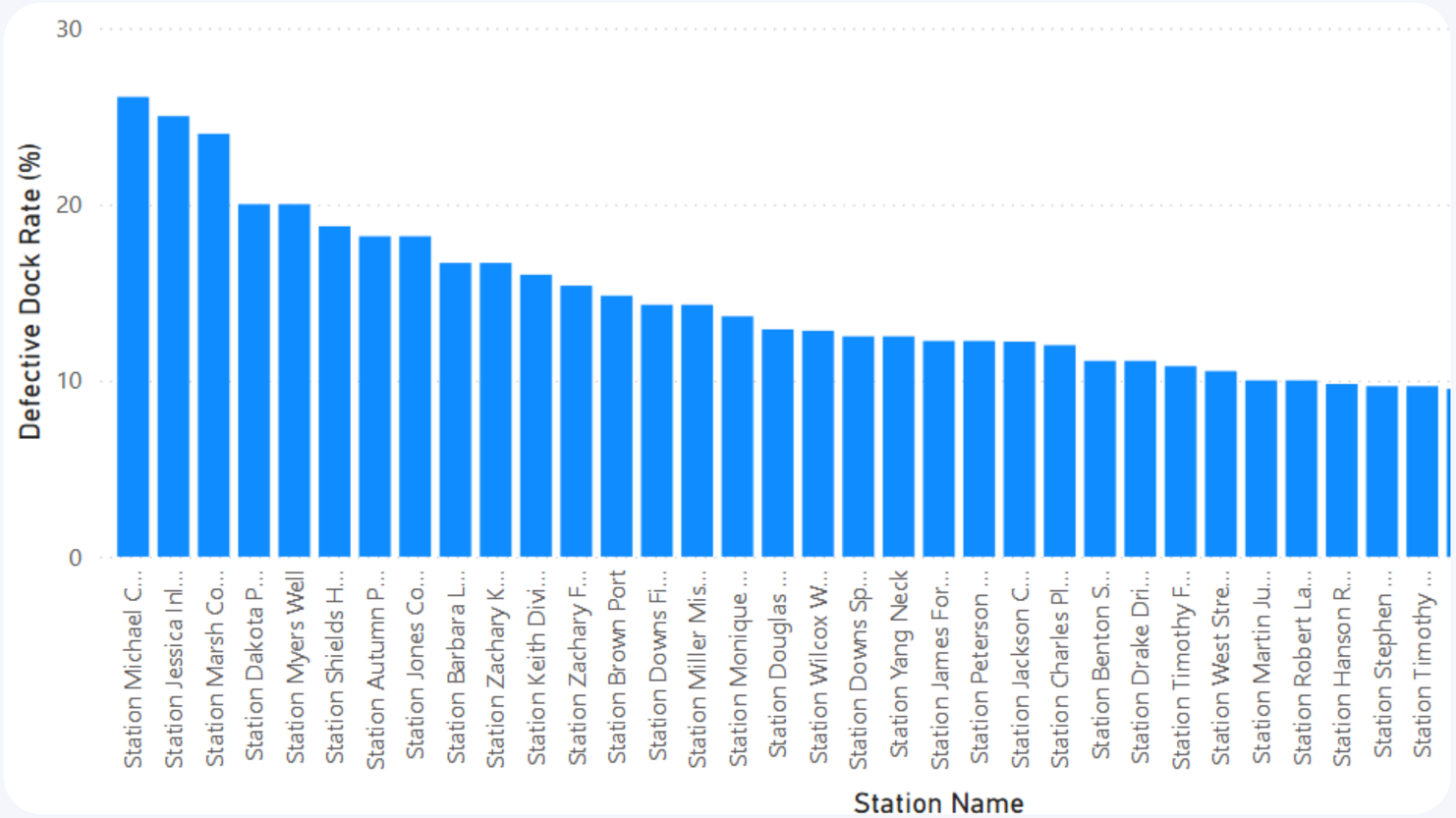
```sql
SELECT
  s.id,                    -- Station ID
  s.name,                  -- Station name

  /* Percentage of defective docks */
  ROUND(
    100.0 * s.defective_docks / s.capacity,
    2
  ) AS defective_dock_rate_percent

FROM stations s

WHERE s.capacity IS NOT NULL
  AND s.capacity > 0
  AND s.defective_docks IS NOT NULL
  AND s.name IS NOT NULL           -- Remove stations

ORDER BY defective_dock_rate_percent DESC;
```



THANK YOU!